ATELIER 4 La première décision

Algorithmie comparative - J2 - Lundi 9 novembre 2026, 14h50 - 90 minutes, en binôme

La question à laquelle l'atelier répond

Quelle variante d'affectation la cellule doit-elle adopter pour que l'étape 2 tienne son budget de nuit en décembre, à 15 000 colis, et jusqu'où cette décision tient-elle ?

Ce que vous produisez : adr/ADR-001-affectation.md, la première des neuf fiches du carnet, avec adr/GRILLE-ADR-001.md, le CSV du harness, celui des couples et la courbe. Ce soir avant 23h59, l'ADR-001 et sa grille sur Moodle ; le carnet complet le 1er décembre.

Le rendu attendu en fin d'atelier

Le correcteur de l'atelier répond sans KO :

```
OK  Les trois variantes sont mesurées avec le harness, échauffement réglé
OK  La courbe de cette mesure est tracée
OK  Les couples colis-livreur sont comptés jusqu'au volume de décembre
OK  Les variantes donnent le même résultat que le legacy (npm test)
OK  La grille comparative de l'ADR-001 est remplie
OK  L'ADR-001 a ses six rubriques, remplies
OK  La rubrique 3 renvoie à une mesure qui existe, et la rubrique 5 donne le domaine de
validité
OK  Les trois phrases pour la directrice des opérations tiennent sans jargon
8 contrôles, 0 KO
```

Pour démarrer

Le premier sous-objectif se fait ensemble, avec l'intervenant, pendant les dix premières minutes. Lisez les deux premières pages pendant les dix premières minutes : la mesure du sous-objectif 1 tourne pendant ce temps.

Sous-objectif 1, mesurer les trois variantes, tracer : la première commande, à la racine du dépôt (le dossier efrei-logistique-perf).

Windows (PowerShell), macOS et Linux

```
npm run bench -- --repetitions 5 --echauffement 200
```

Les sous-objectifs

1. Mesurer les trois variantes, tracer (consigne 1 de la slide)

Pourquoi. Les médianes et la courbe de la rubrique 3 ; la somme de contrôle prouve que les trois variantes font la même chose.

Première action. Lancez `npm run bench -- --repetitions 5 --echauffement 200`, puis, dès qu'il a fini, `npm run courbe -- --log`.

Ce que vous devez voir. La même somme de contrôle pour naive, tri et map à chaque taille. Le mot ARRÊT veut dire que les sommes diffèrent : arrêtez tout.

2. Compter ce que la production paie (consigne 2 de la slide)

Pourquoi. En production, chaque couple colis-livreur examiné coûte une requête à la base ; le comptage n'a pas de chronomètre.

Première action. Lancez `npm run couples`, puis `npm run couples -- --tailles 12000,15000,18000,20000`, et relisez la boucle de legacy/src/affectation.ts.

Ce que vous devez voir. Les nombres de couples de l'annexe exactement, sur tous les postes ; le second comptage sert au domaine de validité de la rubrique 5.

3. Convertir en minutes, noter les critères (consignes 3 et 4 de la slide)

Pourquoi. La note de temps porte sur ce que la production paie, une nuit de décembre à 15 000 colis, face au budget de 43 minutes de l'étape 2.

Première action. Multipliez les couples à 15 000 colis par le coût d'une requête la nuit (1 490 s pour 457 300 requêtes), ajoutez le calcul, puis notez les cinq critères avec les barèmes de l'annexe.

Ce que vous devez voir. Une note de temps par variante ; `npm test` : tous les tests passent ; le cas imposé, un doublon triple, lu dans chaque variante.

4. Remplir la grille, tester la sensibilité (consigne 5 de la slide)

Pourquoi. Les poids du ticket ADR-001 sont imposés : temps 4, mémoire 1, lisibilité 2, robustesse 2, coût 1.

Première action. Copiez adr/grille-comparative.md en adr/GRILLE-ADR-001.md, notez chaque option, puis refaites les totaux avec temps 5, mémoire 1, lisibilité 1, robustesse 2, coût 1.

Ce que vous devez voir. Un chiffre seul, de 1 à 5, par case ; un total pondéré par option ; une ligne de sensibilité.

5. Écrire l'ADR-001, vérifier (consignes 6 et 7 de la slide)

Pourquoi. Six rubriques, une page : c'est la première fiche du carnet noté. En atelier, l'ADR-001 est un premier jet qui passe le correcteur : chaque rubrique en une à trois lignes, avec vos chiffres et vos fichiers. Vous la finissez ce soir, avant 23h59.

Première action. Copiez adr/ADR-000-gabarit.md en adr/ADR-001-affectation.md, remplissez les rubriques dans l'ordre, préparez la question de compréhension, puis lancez `npm run verifier -- 4`.

Ce que vous devez voir. Le correcteur finit sur "8 contrôles, 0 KO" ; vos trois phrases lues à votre binôme, sans main levée.

> **La décision de l'atelier**
> Quelle variante adoptez-vous, et quel critère a tranché ? C'est la rubrique 4 de l'ADR-001, en une phrase, grille jointe ; la rubrique 5 dit dans quel domaine la décision reste vraie.

> **La question de compréhension**
> À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux pour la mise en commun de 16h20.

Les commandes de chaque sous-objectif, pour Windows, macOS et Linux, avec leur sortie attendue, sont dans l'annexe. À 17h00, le binôme commite : git add -A puis git commit -m "J2".

Si ça coince

- `npm run couples` répond "jeu introuvable" : le jeu grande n'est pas généré (`npm run data -- --echelle grande`).
- La grille donne le legacy gagnant : vos notes de temps portent sur les millisecondes du poste, ou les poids ne sont pas ceux du ticket. Relisez le barème du temps dans l'annexe.
- Le correcteur refuse les trois phrases : il nomme le mot technique qu'il a trouvé ; reformulez à partir du monde de Nadia (des livreurs, des colis, des minutes).

Toujours bloqué après ces trois pistes : levez la main, ou écrivez sur Teams. D'autres cas sont à la fin de l'annexe.

Si vous avez fini

- Cherchez le poids du temps à partir duquel la décision bascule, les autres poids gardés : une phrase dans la grille.
- Écrivez ce qui se passe si l'étape 2 tourne le matin, à 14,8 ms la requête : la décision change-t-elle ?

Où trouver un exemple dans le code

adr/ADR-000-gabarit.md donne les six rubriques ; l'annexe de cette fiche donne une rubrique 6 complète sur un autre sujet. bench/couples.ts montre comment les couples sont comptés.

Annexe : toutes les commandes, dans l'ordre

Toutes les commandes se lancent à la racine du dépôt. Les sorties montrées sont de vraies sorties, capturées sous Windows en jouant cette fiche ; vos durées seront différentes, vos nombres de colis, de couples et vos sommes de contrôle non. "Extrait" signale qu'on n'en montre que les lignes utiles.

1. Mesurer les trois variantes, tracer

Windows (PowerShell), macOS et Linux

```
npm run bench -- --repetitions 5 --echauffement 200
```

Sur le poste de capture : 39 secondes, surtout pour naive à 10 000 colis ; la chauffe est plafonnée à 5 secondes par variante et par taille.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
[harness] variante=naive taille=10000 repetition=1 duree_ms=1392.88 memoire_ko=3789
controle=058848ce
[harness] variante=tri taille=10000 repetition=1 duree_ms=18.02 memoire_ko=34314
controle=058848ce
[harness] variante=map taille=10000 repetition=1 duree_ms=6.06 memoire_ko=5204
controle=058848ce
variante        taille   duree_ms   memoire_ko
naive             1000      17.82         1092
naive             2000      62.75         2214
naive             5000     386.59         8287
naive            10000    1401.43         3789
tri               1000       2.51         3489
tri               2000       4.75         6946
tri               5000      10.61        17282
tri              10000      17.75        34314
map               1000       0.56          517
map               2000       1.02         1021
map               5000       2.89         2645
map              10000       6.45         5204
```

Windows (PowerShell), macOS et Linux

```
npm run courbe -- --log
```

Juste après la mesure : sans chemin, la commande trace le CSV le plus récent de bench/resultats. Après npm run couples, elle refuse celui des couples (colonnes attendues : variante;taille;duree_ms) : donnez-lui alors le chemin du CSV d'affectation.

Ce que vous devez voir (sortie réelle, Windows) :

```
fichier : bench\resultats\2026-11-08T17-26-44-affectation.csv
  naive      taille=   1000  mediane_ms=      17.8  (5 mesures)
  naive      taille=   2000  mediane_ms=      62.7  (5 mesures)
  naive      taille=   5000  mediane_ms=     386.6  (5 mesures)
  naive      taille=  10000  mediane_ms=    1401.4  (5 mesures)
  tri        taille=   1000  mediane_ms=       2.5  (5 mesures)
  tri        taille=   2000  mediane_ms=       4.8  (5 mesures)
  tri        taille=   5000  mediane_ms=      10.6  (5 mesures)
  tri        taille=  10000  mediane_ms=      17.8  (5 mesures)
  map        taille=   1000  mediane_ms=       0.6  (5 mesures)
  map        taille=   2000  mediane_ms=       1.0  (5 mesures)
  map        taille=   5000  mediane_ms=       2.9  (5 mesures)
  map        taille=  10000  mediane_ms=       6.4  (5 mesures)
courbe écrite : bench\resultats\2026-11-08T17-26-44-affectation.svg
```

2. Compter ce que la production paie

Windows (PowerShell), macOS et Linux

```
npm run couples
```

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
[couples] jeu=grande livreurs=85 hubs=3 tailles=6412,9000,15000
[couples] variante=tri taille=15000 doublons=38 non_affectes=178 couples=1271770
couples_par_colis=84.8 controle=6d6f2a8c
[couples] variante=map taille=15000 doublons=38 non_affectes=178 couples=651031
couples_par_colis=43.4 controle=6d6f2a8c
```

Windows (PowerShell), macOS et Linux

```
npm run couples -- --tailles 12000,15000,18000,20000
```

Même format de sortie. Les couples se comptent sans chronomètre : identiques sur tous les postes. À 12 000, 15 000, 18 000 et 20 000 colis, tri : 1 017 450, 1 271 770, 1 525 835 et 1 694 645 ; map : 456 335, 651 031, 905 096 et 1 073 906 couples ; la ligne taille=15000 est celle de la commande précédente. Ces volumes servent à trouver où chaque variante sort du budget (annexe, le domaine de validité).

Windows (PowerShell)

```
code legacy\src\affectation.ts
```

macOS et Linux

```
code legacy/src/affectation.ts
```

Le legacy ne se modifie pas : sa boucle se lit.

3. Convertir en minutes, noter les critères

Windows (PowerShell), macOS et Linux

```
npm test
```

La robustesse commence là ; puis le cas imposé de l'annexe, un doublon triple.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
✔ les variantes tri et map donnent le résultat du legacy (32.6659ms)
✔ un lot vide donne des tournées vides (1.6392ms)
ℹ tests 2
ℹ pass 2
ℹ fail 0
```

4. Remplir la grille, tester la sensibilité

Windows (PowerShell)

```
Copy-Item adr\grille-comparative.md adr\GRILLE-ADR-001.md
```

macOS et Linux

```
cp adr/grille-comparative.md adr/GRILLE-ADR-001.md
```

Une note est un chiffre seul, de 1 à 5, par case : "4", pas "4/5" ni "4 (lisible)". Trois options : A le legacy, B tri, C map.

5. Écrire l'ADR-001, vérifier

Windows (PowerShell)

```
Copy-Item adr\ADR-000-gabarit.md adr\ADR-001-affectation.md
```

macOS et Linux

```
cp adr/ADR-000-gabarit.md adr/ADR-001-affectation.md
```

Rubrique 3 : vos fichiers de bench/resultats ; dans le tableau, la colonne 100 000 ne se remplit que pour le legacy, avec la médiane de l'atelier 2 (bench/resultats/j1-chrono.csv), le harness s'arrête à 10 000 colis. Rubrique 4 : une phrase. Rubrique 5 : le domaine de validité de l'annexe, avec un nombre. Rubrique 6 : trois phrases sans un mot de la liste de l'annexe.

Windows (PowerShell), macOS et Linux

```
npm run verifier -- 4
```

Le correcteur vérifie que l'ADR est remplie et que les trois phrases n'ont pas de jargon ; il ne dit pas si la décision est bonne : c'est la relecture de 16h20.

Ce que vous devez voir (extrait de la sortie réelle, Windows) :

```
OK  Les trois variantes sont mesurées avec le harness, échauffement réglé
OK  La courbe de cette mesure est tracée
OK  Les couples colis-livreur sont comptés jusqu'au volume de décembre
OK  Les variantes donnent le même résultat que le legacy (npm test)
OK  La grille comparative de l'ADR-001 est remplie
OK  L'ADR-001 a ses six rubriques, remplies
OK  La rubrique 3 renvoie à une mesure qui existe, et la rubrique 5 donne le domaine de
validité
OK  Les trois phrases pour la directrice des opérations tiennent sans jargon
8 contrôles, 0 KO
```

Les barèmes, tels que le cours 2 les pose

Temps (poids 4) : les minutes de nuit à 15 000 colis, couples x coût d'une requête la nuit (3,26 ms), plus le calcul, face au budget de 43 minutes. 5 sous 20 minutes, 4 sous 43, 3 sous 65, 2 sous 90, 1 au-delà. Le harness s'arrête à 10 000 colis : le calcul du legacy à 15 000 s'extrapole, quadratique, sa médiane à 10 000 x 2,25 (3,2 s sur le poste de capture) ; celui de tri et de map reste sous la seconde. Les deux pèsent peu devant les requêtes, qui se comptent en dizaines de minutes.

Mémoire (poids 1) : le tas relevé par le harness à 10 000 colis, lu comme un ordre de grandeur. Deux options à moins d'un facteur 2 l'une de l'autre ont la même note. Relancez la mesure une fois : si un relevé bouge d'un facteur 2 ou plus entre les deux lancements, la mémoire ne départage pas, et les trois options prennent la même note.

Lisibilité (poids 2) : la meilleure mesure reste de faire lire son code à quelqu'un qui ne l'a pas écrit. Ici, c'est votre cas : lisez les trois variantes de bench/variantes/affectation.ts (naive renvoie au legacy) et répondez, pour chacune, à trois questions : que fait la fonction, que se passe-t-il sur un doublon, où changer la règle "pharmacies d'abord". Listez les idées qu'il faut connaître pour répondre ; on part de 5 et chaque idée (un Set, une Map, un tri par index) coûte un point.

Robustesse (poids 2) : `npm test` d'abord, les variantes donnent-elles le résultat du legacy ? Puis le cas imposé, un doublon triple (le même colis trois fois) : lisez comment chaque variante écarte un doublon ; elle doit affecter le colis une fois et compter deux doublons. Barème : 5 si les doublons, les fichiers vides et les entrées invalides sont traités ou refusés par une erreur claire ; 3 si seul le doublon est traité ; 2 si le doublon triple donne un résultat faux en silence ; 1 si `npm test` échoue.

Coût de mise en oeuvre (poids 1) : les lignes changées par rapport au legacy, les dépendances ajoutées, les données à préparer et à maintenir (un index, un fichier trié, un cache). Barème : 5 si rien ne change ; 3 pour quelques lignes et une donnée à préparer à chaque lancement ; 2 pour une structure nouvelle dans l'équipe ; 1 si une dépendance s'ajoute au projet.

Le temps, la lisibilité, la robustesse et le coût ont chacun leur barème ci-dessus ; la mémoire se note avec la règle du facteur 2. Dans tous les cas, justifiez la note en une phrase sous la grille : le barème dit la note, la phrase dit ce que vous avez observé.

Le domaine de validité (rubrique 5)

Une décision vaut pour des paramètres : le volume, le nombre de livreurs, le nombre de hubs, l'heure. Écrivez ce que vous avez fait varier (le volume, avec le second comptage de couples), ce que vous n'avez pas fait varier, et le volume à partir duquel la décision ne tient plus, avec un nombre.

Le volume limite d'une variante : le premier volume où ses minutes de nuit dépassent le budget de 43 minutes. Avec quatre volumes, donnez l'intervalle qui l'encadre, ou interpolez.

Les deux autres paramètres, mesurés en préparant le module : sous une trentaine de livreurs, tri calcule plus vite que map ; avec un hub par livreur, l'avantage de map disparaît. Efrei a 85 livreurs et 3 hubs.

La rubrique 6 : les mots interdits et un exemple

Les mots interdits : algorithme, complexité, quadratique, O(n), boucle, tableau, indexOf, cache, référentiel, index, requête, base de données, variante, benchmark, performance, harness, map, Set, médiane, milliseconde, ms. Si vous hésitez sur un mot, il est technique.

Un exemple complet, sur une autre décision (stocker les jeux de données en CSV plutôt qu'en JSON) : "Nos fichiers de test se lisent comme une feuille Excel, une ligne par colis. On les a choisis parce que n'importe qui peut les ouvrir et les relire, et qu'ils prennent deux fois moins de place. Ils se chargent en deux secondes ; l'autre format aussi, donc la vitesse n'a pas décidé."

Ce qui le rend valable : aucun mot à chercher, une comparaison de son monde, un chiffre qu'elle peut répéter, la raison de la décision et pas seulement la décision.

Cas particuliers

- `npm run couples` s'arrête sur "la copie comptée ne donne pas le résultat de la variante mesurée" : bench/variantes/affectation.ts a été modifié ; revenez à la version du dépôt.
- `npm run courbe` répond "colonnes attendues : variante;taille;duree_ms" : elle a été lancée après `npm run couples`. Donnez-lui le chemin du CSV d'affectation : `npm run courbe -- bench/resultats/<horodatage>-affectation.csv --log`.
- Votre harness ne tourne pas encore : commencez par `npm run couples`, qui n'en dépend pas, et appelez l'intervenant.
- Le correcteur ne compte pas vos options notées : une case contient autre chose qu'un chiffre de 1 à 5.